nomic-ai/nomic-embed-text-v1.5

like 901 | Follow Nomic AI 1.22k

Sentence Similarity | sentence-transformers | ONNX | Safetensors | Transformers | Transformers.js | English | nomic_bert | feature-extraction | mteb | custom_code | Eval Results (legacy) | Eval Results | text-embeddings-inference | arxiv:2402.01613 | arxiv:2205.13147 | License: apache-2.0

Deploy | Copy to bucket NEW | Use this model

Model card | Files xet | Community 61

nomic-embed-text-v1.5: Resizable Production Embeddings with Matryoshka Representation Learning

Blog | Technical Report | AWS SageMaker | Nomic Platform

Exciting Update!: `nomic-embed-text-v1.5` is now multimodal! nomic-embed-vision-v1.5 is aligned to the embedding space of `nomic-embed-text-v1.5`, meaning any text embedding is multimodal!

Usage

Important: the text prompt *must* include a *task instruction prefix*, instructing the model which task is being performed.

For example, if you are implementing a RAG application, you embed your documents as `search_document: <text here>` and embed your user queries as `search_query: <text here>`.

Downloads last month
16,471,818

Safetensors

Model size | 0.1B params
Tensor type | F32 | Files info

Inference Providers NEW

Sentence Similarity

This model isn't deployed by any Inference Provider.

3 Ask for provider support

Model tree for nomic-ai/nomic-em...

Finetunes 40 models
Quantizations 36 models

Spaces using nomic-ai/nomic... 100

mteb/leaderboard

Notice: From transformers v5.5.0 and sentence transformers v5.3.0, `trust_remote_code=True` will no longer be necessary. This will only be possible with the text-only series as of now.

Task instruction prefixes

`search_document`

Purpose: embed texts as documents from a dataset

This prefix is used for embedding texts as documents, for example as documents for a RAG index.

```
from sentence_transformers import SentenceTra

model = SentenceTransformer("nomic-ai/nomic-
sentences = ['search_document: TSNE is a dim
embeddings = model.encode(sentences)
print(embeddings)
```

`search_query`

Purpose: embed texts as questions to answer

This prefix is used for embedding texts as questions that documents from a dataset could resolve, for example as queries to be answered by a RAG application.

```
from sentence_transformers import SentenceTra

model = SentenceTransformer("nomic-ai/nomic-
sentences = ['search_query: Who is Laurens va
embeddings = model.encode(sentences)
print(embeddings)
```

hakari-bench/leaderboard

eduagarcia/multilingual-tokenizer-le...

mteb/arena

Xenova/adaptive-retrieval-web

mteb/leaderboard_legacy

Xenova/webgpu-nomic-embed

reddit-tools-HF/nomic-embeddings

+ 92 Spaces

Collection including `nomic-ai/nom...`

Nomic Embed Collection
Open Sour... • 8 items • Upd... • △ 24

Papers for `nomic-ai/nomic-embed...`

Nomic Embed: Training a Reproduc...
Paper • 2402.01613 • Publi... • △ 18

Matryoshka Representation Learning
Paper • 2205.13147 • Publi... • △ 30

Evaluation results

mteb/arguana

ArguAna Default ...	source		52.02*
ArguAna	source		52.02*
accuracy ...	test set	self-reported	75.209
ap on MTE...	test set	self-reported	38.576
f1 on MTE...	test set	self-reported	69.356
accuracy ...	test set	self-reported	91.814
ap on MTE...	test set	self-reported	88.652

Expand 985 evaluations

clustering

Purpose: embed texts to group them into clusters

This prefix is used for embedding texts in order to group them into clusters, discover common topics, or remove semantic duplicates.

```
from sentence_transformers import SentenceTra

model = SentenceTransformer("nomic-ai/nomic-
sentences = ['clustering: the quick brown fo
embeddings = model.encode(sentences)
print(embeddings)
```

classification

Purpose: embed texts to classify them

This prefix is used for embedding texts into vectors that will be used as features for a classification model

```
from sentence_transformers import SentenceTra

model = SentenceTransformer("nomic-ai/nomic-
sentences = ['classification: the quick brow
embeddings = model.encode(sentences)
print(embeddings)
```

Sentence Transformers

```
import torch.nn.functional as F
from sentence_transformers import SentenceTra

matryoshka_dim = 512

model = SentenceTransformer("nomic-ai/nomic-
```

```
sentences = ['search_query: What is TSNE?',
embeddings = model.encode(sentences, convert
embeddings = F.layer_norm(embeddings, normal
embeddings = embeddings[:, :matryoshka_dim]
embeddings = F.normalize(embeddings, p=2, di
print(embeddings)
```

Transformers

```
import torch
import torch.nn.functional as F
from transformers import AutoTokenizer, Auto

def mean_pooling(model_output, attention_mas
    token_embeddings = model_output[0]
    input_mask_expanded = attention_mask.uns
    return torch.sum(token_embeddings * inpu

sentences = ['search_query: What is TSNE?',

tokenizer = AutoTokenizer.from_pretrained('b
model = AutoModel.from_pretrained('nomic-ai/
model.eval()

encoded_input = tokenizer(sentences, padding

+ matryoshka_dim = 512

with torch.no_grad():
    model_output = model(**encoded_input)

embeddings = mean_pooling(model_output, enco
+ embeddings = F.layer_norm(embeddings, norm
+ embeddings = embeddings[:, :matryoshka_dim
embeddings = F.normalize(embeddings, p=2, di
print(embeddings)
```

The model natively supports scaling of the sequence length past 2048 tokens. To do so,

```
- tokenizer = AutoTokenizer.from_pretrained(
+ tokenizer = AutoTokenizer.from_pretrained(

- model = AutoModel.from_pretrained('nomic-a
+ rope_parameters = {"rope_theta": 1000.0, "
+ model = AutoModel.from_pretrained('nomic-a
```

Transformers.js

```
import { pipeline, layer_norm } from '@huggi

// Create a feature extraction pipeline
const extractor = await pipeline('feature-ex

// Define sentences
const texts = ['search_query: What is TSNE?'

// Compute sentence embeddings
let embeddings = await extractor(texts, { po
console.log(embeddings); // Tensor of shape

const matryoshka_dim = 512;
embeddings = layer_norm(embeddings, [embeddi
    .slice(null, [0, matryoshka_dim])
    .normalize(2, -1);
console.log(embeddings.tolist());
```

Nomic API

The easiest way to use Nomic Embed is through the Nomic Embedding API.

Generating embeddings with the `nomic` Python client is as easy as

```
from nomic import embed

output = embed.text(
```

```
    texts=['Nomic Embedding API', '#keepAIOp
    model='nomic-embed-text-v1.5',
    task_type='search_document',
    dimensionality=256,
)

print(output)
```

For more information, see the API reference

Infinity

Usage with Infinity.

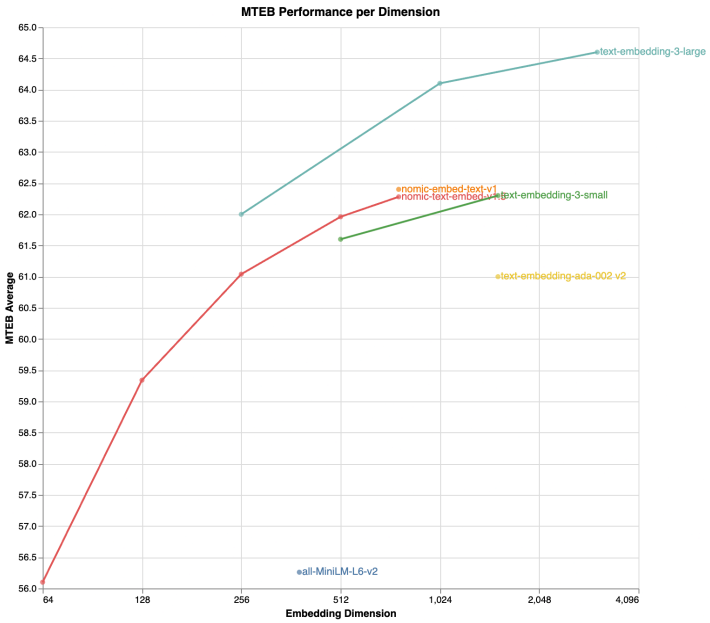
```
docker run --gpus all -v $PWD/data:/app/.cach
michaelf34/infinity:0.0.70 \
v2 --model-id nomic-ai/nomic-embed-text-v1.5
```

Adjusting Dimensionality

`nomic-embed-text-v1.5` is an improvement upon Nomic Embed that utilizes Matryoshka Representation Learning which gives developers the flexibility to trade off the embedding size for a negligible reduction in performance.

Name	SeqLen	Dimension	MTEB
nomic-embed-text-v1	8192	768	**62.39**
nomic-embed-text-v1.5	8192	768	62.28
nomic-embed-text-v1.5	8192	512	61.96
nomic-embed-text-v1.5	8192	256	61.04
nomic-embed-text-v1.5	8192	128	59.34

Name	SeqLen	Dimension	MTEB
nomic-embed-text-v1.5	8192	64	56.10



Training

Click the Nomic Atlas map below to visualize a 5M sample of our contrastive pretraining data!

We train our embedder using a multi-stage training pipeline. Starting from a long-context BERT model, the first unsupervised contrastive stage trains on a dataset generated from weakly related text pairs, such as question-answer pairs from forums like StackExchange and Quora, title-body pairs from Amazon reviews, and summarizations from news articles.

In the second finetuning stage, higher quality labeled datasets such as search queries and answers from web searches are leveraged. Data curation and hard-example mining is crucial in this stage.

For more details, see the Nomic Embed Technical Report and corresponding blog post.

Training data to train the models is released in its entirety. For more details, see the `contrastors` repository

Join the Nomic Community

- Nomic: https://nomic.ai
- Discord: https://discord.gg/myY5YDR8z8
- Twitter: https://twitter.com/nomic_ai

Citation

If you find the model, dataset, or training code useful, please cite our work

```
@misc{nussbaum2024nomic,
      title={Nomic Embed: Training a Reprodu
      author={Zach Nussbaum and John X. Morr
      year={2024},
      eprint={2402.01613},
```

```
      archivePrefix={arXiv},
      primaryClass={cs.CL}
}
```